

Федеральное государственное автономное образовательное учреждение
высшего образования

«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Направление подготовки 09.03.04 «Программная инженерия» –

Системное и прикладное программное обеспечение

Дисциплина: Рефакторинг баз данных и приложений

Этап 1: Подготовка инфраструктуры

Выполнили:

студенты 4 курса

Кравцов Кирилл Денисович

Группа: Р3411

Батманов Даниил Евгеньевич

Группа: Р3407

Преподаватель:

Логинов Иван Павлович

г. Санкт-Петербург, 2025

1.1 Вынесение OpenAPI из кода

[Pull request](#)

Задача

Проблема: Документация API генерируется только при запущенном приложении, что затрудняет доступ к спецификации для внешних разработчиков и клиентов API.

Цель:

- Сделать API документацию доступной 24/7 без запуска приложения
- Предоставить публичный доступ к спецификации API
- Отделить документацию от runtime-окружения

Реализация: Экспортировать OpenAPI спецификацию в статические файлы (YAML и HTML) и опубликовать на GitHub Pages.

Результаты

Документация API извлечена из кода приложения и опубликована в виде статических файлов на GitHub Pages. Это обеспечивает постоянный доступ к спецификации без необходимости запуска приложения.

Что было сделано:

- Экспортирована OpenAPI спецификация в docs/api/api-docs.yaml
- Создана HTML-версия Swagger UI в docs/api/api-docs.html
- Документация опубликована на GitHub Pages:
- [HTML \(Swagger UI\)](#)
- [YAML \(спецификация\)](#)

1.2. Вынесение миграций из приложения

[Pull request](#)

Задача

Проблема: Миграции Flyway выполняются при старте приложения, что создает зависимость между развертыванием приложения и миграциями БД.

Цель:

- Разделить ответственность: приложение и миграции БД должны запускаться независимо
- Обеспечить явный контроль над процессом миграций
- Избежать проблем при масштабировании (несколько экземпляров приложения)
- Реализация: Добавить отдельный сервис Flyway в docker-compose.yml для выполнения миграций до запуска приложения.

Результаты

Миграции базы данных отделены от процесса запуска приложения и вынесены в отдельный сервис Flyway в Docker Compose. Это обеспечивает независимое управление схемой БД и позволяет избежать проблем при масштабировании.

Что было сделано:

- Создан отдельный сервис Flyway в docker-compose.yml
- Flyway выполняет миграции до запуска приложения через depends_on
- Удалены зависимости Flyway из приложения
- Приложение теперь только подключается к уже мигрированной БД

Принесённая ценность:

- Разделение ответственности: Управление схемой БД отделено от логики приложения
- Независимое управление: Миграции можно запускать отдельно от развертывания приложения
- Безопасность при масштабировании: Исключены конфликты при одновременном запуске нескольких экземпляров приложения
- Сокращенное время запуска: Приложение запускается быстрее, не выполняя проверку и применение миграций
- Явный контроль: Видно, что миграции выполнены до старта приложения

1.3. Миграция системы сборки на Gradle

[Pull request](#)

Задача

Проблема: Использование Maven с XML-конфигурацией, которая не будет гармонично сочетаться с последующим переходом на Kotlin.

Цель:

- Ускорить процесс сборки (incremental compilation, build cache)
- Получить типобезопасную конфигурацию сборки через Kotlin DSL
- Упростить управление зависимостями через version catalogs
- Подготовиться к переходу на Kotlin (единая экосистема инструментов)
- Реализация: Заменить Maven на Gradle, используя Kotlin DSL для конфигурации сборки и version catalogs для управления зависимостями.

Результаты

Система сборки перенесена с Maven на Gradle с Kotlin DSL. Это обеспечивает типобезопасную конфигурацию сборки, ускоренные инкрементальные сборки и лучшую интеграцию с последующим переходом на Kotlin.

Что было сделано:

- Создан build.gradle.kts с Kotlin DSL вместо pom.xml
- Создан settings.gradle.kts для настройки проекта
- Внедрен gradle/libs.versions.toml для централизованного управления версиями зависимостей
- Настроены плагины: Spring Boot, Kotlin (JVM, Spring, JPA), Kapt (для MapStruct)
- Добавлен Gradle wrapper (gradlew, gradlew.bat) для воспроизводимыхборок

- Удалены Maven-файлы: pom.xml, mvnw, mvnw.cmd, .mvn/

Принесённая ценность:

- Производительность: Incremental compilation и build cache значительно ускоряют повторные сборки
- Типобезопасность: Kotlin DSL обеспечивает типобезопасную конфигурацию сборки с поддержкой IDE
- Version Catalogs: Централизованное управление версиями упрощает обновление зависимостей
- Гибкость: Rich API Gradle позволяет создавать сложную логику сборки при необходимости
- Подготовка к Kotlin: Единая экосистема инструментов (Kotlin DSL + Kotlin код)
- Воспроизводимость: Gradle wrapper обеспечивает одинаковые сборки на всех машинах

Презентация

- смотреть видео “Демонстрация_работающего_приложения”
- смотреть видео “Обзор_выполненной_работы_код,_документация”